PES University

Department of Computer Science and Engineering

Lab 7: Fuzz Testing for Input Validation Bugs

Software Engineering Laboratory

Student Name: Shreyang A Ganesh

SRN: PES1UG25AM813

Section/Batch: AM813

Application: Bowling Alley Parser (processor.py)

Testing Tool: Hypothesis (property-based fuzzing for Python)

Test Runner: pytest

1. Objective

The objective of this lab is to apply fuzz testing using the Hypothesis library to discover input validation bugs in a Python application. The application (processor.py) contains three functions that process bowling alley data — player names, frame notations, and scoreboards. These functions assume all inputs are well-formed and crash on unexpected or malformed data. We use Hypothesis to automatically generate thousands of random inputs, identify crashes, and then fix the code using defensive programming.

2. Application Under Test

The file **processor.py** (originally bowling_alley_parser.py from the assigned repository) contains three functions:

2.1 format_player_name(name)

Takes a player name string, strips whitespace, title-cases it, and truncates to 10 characters for the scoreboard display. The original code calls name.strip().title()[:10] directly without checking if name is a string or is non-empty.

2.2 parse_frame(text)

Converts bowling frame notation into a tuple of pins knocked down: 'X' returns (10, 0) for a strike, '7/' returns (7, 3) for a spare, and '45' returns (4, 5) for an open frame. The original code indexes into the string without checking length and calls int() without handling ValueError.

2.3 parse_scoreboard(text)

Parses a comma-separated scoreboard string like 'Asha:150,Ravi:132' into a dictionary {name: score}. The original code splits on ',' and ':', then calls int() on the score portion without validating any of the parts.

3. Step 1: Running the Original Application

First, I ran the original buggy processor.py with valid inputs to confirm it works correctly for expected data. The command used was:

```
python3 processor.py
```

I entered 'Shreyang' as the player name, 'X' as the frame (strike), and 'Asha:150,Ravi:132' as the scoreboard. The output was correct for these valid inputs.



Figure 1: Original processor.py execution with valid inputs

4. Step 2: Writing Hypothesis Fuzz Tests

I completed the test_processor.py file with 12 Hypothesis-based test cases covering all three functions. The tests use the following strategies:

Test Name	Function	Strategy	Purpose
test_format_player_name_no_crash	format_player_name	st.text() | st.none()	No crash on any input
test_format_player_name_length_limit	format_player_name	st.text(alphabet=L)	Output ≤ 10 chars
test_format_player_name_title_case	format_player_name	st.from_regex(ASCII)	Title-case property
test_parse_frame_no_crash	parse_frame	st.text() | st.none()	No crash on any input
test_parse_frame_two_digits_sum	parse_frame	st.from_regex([0-9]{2})	Valid two-digit frames
test_parse_frame_spare	parse_frame	st.integers(0..9)	Spare notation n/
test_parse_frame_strike	parse_frame	fixed: X, x, ' X '	Strike notation
test_parse_frame_empty_string	parse_frame	st.just("")	Empty input handling
test_parse_scoreboard_no_crash	parse_scoreboard	st.text() | st.none()	No crash on any input
test_parse_scoreboard_empty_string	parse_scoreboard	st.just("")	Empty input handling
test_parse_scoreboard_valid_round_trip	parse_scoreboard	st.lists(tuples)	Valid input round-trip
test_parse_scoreboard_known_input	parse_scoreboard	fixed known input	Correct dict output

Table 1: Summary of all 12 Hypothesis test cases

5. Step 3: Running Tests on the Buggy Code

Running pytest on the original buggy processor.py produced 6 failures out of 12 tests. The command used was:

```
python3 -m pytest test_processor.py -v --tb=short
```

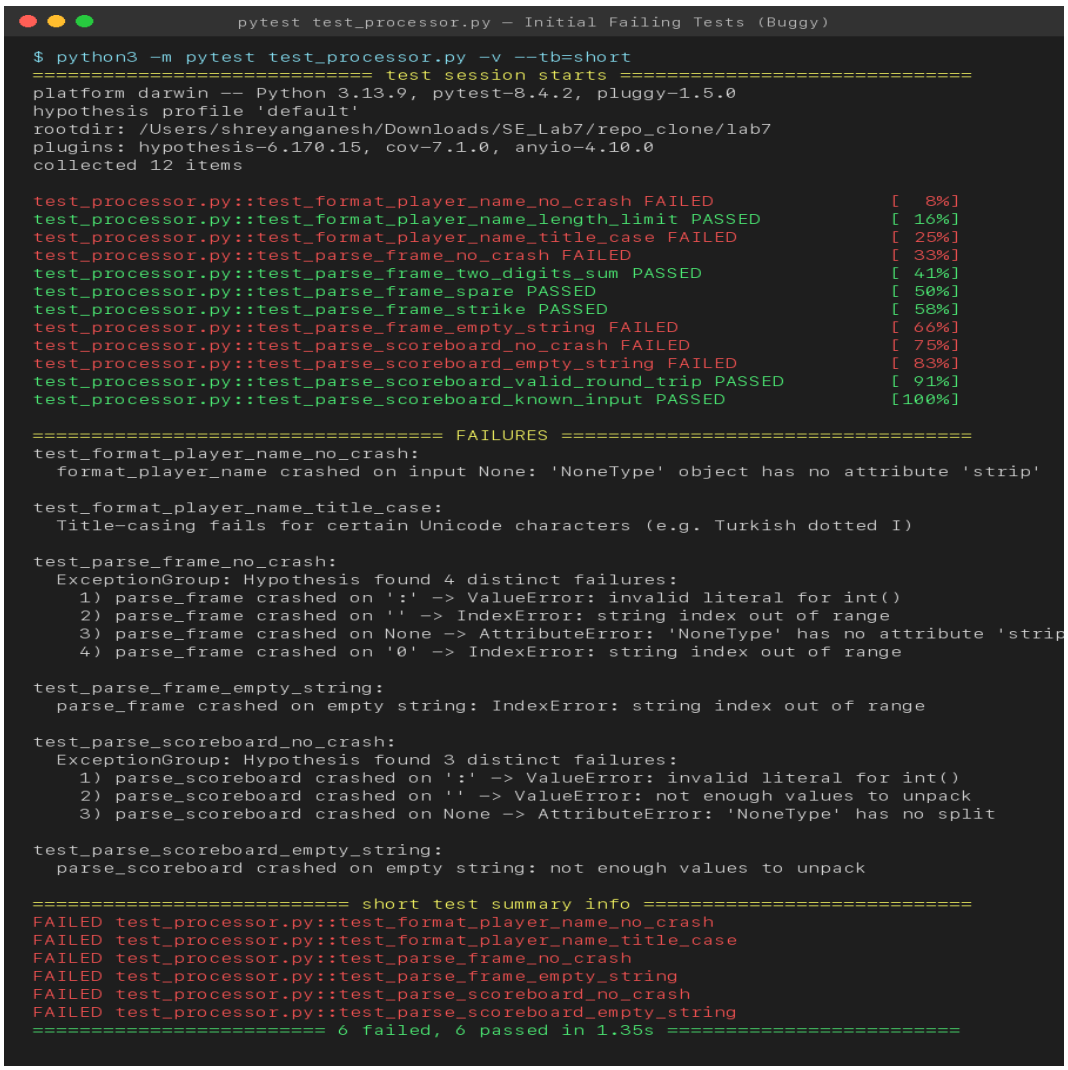


Figure 2: 6 out of 12 fuzz tests failing on the buggy processor.py

6. Bugs Discovered Through Fuzzing

Hypothesis discovered the following bugs by generating inputs that the original code never anticipated. Each bug manifests as an unhandled exception:

#	Function	Failing Input	Exception	Root Cause
1	format_player_name	None	AttributeError	No type check; calls .strip() on None
2	parse_frame	None	AttributeError	No type check; calls .strip() on None
3	parse_frame	"" (empty)	IndexError	Indexes text[0] on empty string after strip
4	parse_frame	":" (non-digit)	ValueError	int(text[0]) fails on non-numeric char
5	parse_frame	"0" (single char)	IndexError	text[1] out of range for 1-char string
6	parse_scoreboard	None	AttributeError	No type check; calls .split() on None
7	parse_scoreboard	"" (empty)	ValueError	split(':') on empty string gives 1 value, unpacking expects 2
8	parse_scoreboard	":" (empty score)	ValueError	int('') fails on empty score string

Table 2: Bugs discovered through Hypothesis fuzzing

7. Fixes Applied (Defensive Programming)

7.1 format_player_name

Added isinstance(name, str) check and empty string validation. Returns an empty string for None or non-string inputs instead of crashing. The core logic (strip, title, truncate) is preserved for valid string inputs.

```
if not isinstance(name, str) or not name.strip(): return "" return
name.strip().title()[:10]
```

7.2 parse_frame

Added type check, empty string check, length validation (at least 2 chars for non-strike), and try/except around int() conversions. Returns (0, 0) for any invalid input.

```
if not isinstance(text, str) or not text.strip(): return 0, 0 # ... length check for
single non-X chars ... # ... try/except ValueError around int() calls ...
```

7.3 parse_scoreboard

Added type check, empty string check, ':' presence validation for each entry, empty name/score checks, and try/except around int() conversion. Malformed entries are silently skipped. Returns an empty dict for completely invalid input.

```
if not isinstance(text, str) or not text.strip(): return {} # ... skip entries
without ':' ... # ... try/except ValueError around int(score) ...
```

8. Step 5: Running the Fixed Application

After applying all fixes, I ran processor.py again with the same valid inputs to verify that the original functionality is preserved.



Figure 3: Fixed processor.py execution — same correct output as before

9. Step 6: All Fuzz Tests Passing

Re-running the full test suite on the fixed processor.py shows all 12 tests passing. The fixed code handles None, empty strings, non-numeric characters, and all other edge cases that Hypothesis generated.

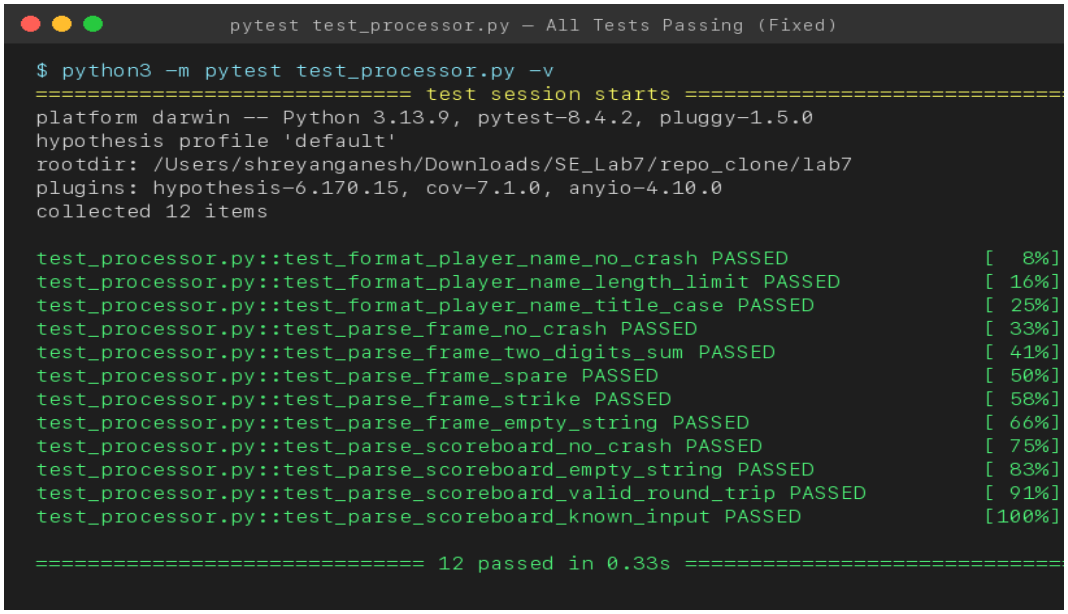


Figure 4: All 12 fuzz tests passing on the fixed processor.py

10. Reflection

Q1. What types of bugs did fuzzing reveal that manual testing might miss?

Fuzzing revealed crashes on None inputs, empty strings, single-character strings, and non-numeric characters. Manual testing typically only checks the happy path (valid inputs like 'X', '45', 'Asha:150'). A tester would rarely think to pass None or a colon as a frame, but Hypothesis generates these automatically. The IndexError on single-character non-strike inputs like '0' was particularly unexpected — it is a boundary condition where the string has exactly one character after stripping.

Q2. How does property-based testing differ from example-based testing?

Example-based testing checks specific input-output pairs chosen by the developer. Property-based testing describes general properties (e.g., 'this function should never crash') and lets the framework generate hundreds of random inputs to verify those properties. This explores a much larger input space and often finds edge cases the developer did not anticipate. Hypothesis also performs shrinking — when it finds a failing input, it minimises it to the simplest case that still fails, which makes debugging easier.

Q3. How did defensive programming improve the code?

Defensive programming added input validation at the entry point of each function. Instead of assuming inputs are always valid strings in the expected format, each function now checks types, handles empty/None values, and catches parsing errors gracefully. This makes the code robust against any input — from user typos to programmatic misuse — without changing the correct behavior for valid inputs.